

Développement d'application mobile

Analyse et conception

Rapport

Gabriel Bolduc

Gabriel Racine

Problème

Aujourd'hui, les plateformes de streaming offrent énormément de choix de film ou série. Et c'est justement ça le problème, il y a trop de choix. Que ce soit seul, en couple ou entre amis, on passe souvent plus de temps à chercher qu'à regarder. Cette abondance de choix rend la décision difficile.

Notre solution

Nous proposons un réseau social conçu pour faciliter le partage d'avis sur les films entre amis, afin de rendre le choix d'un film plus simple. Notre application permet : (Modules)

Usagers :

- Inscription
- Connexion / Déconnexion
- Gestion de profil (Changer photo, gérer le dark mode)

Amis (Follow) :

- Create (Suivre un ami)
- Read (Liste d'ami)
- Delete (Supprimer un ami)

Ratings :

- Create (Noter un film)
- Read (Afficher les évaluations sur le feed et ma liste d'évaluations)
- Update (Modifier une évaluation)
- Delete (Supprimer évaluation)
- List (Lister évaluations)

Pourquoi c'est mieux que ce qui existe déjà

Notre application met les avis de nos amis en priorité. Sur les plateformes déjà existantes, on voit surtout des critiques de cinéphiles ou de journalistes. C'est souvent trop technique, trop sérieux, ou pas pertinent pour quelqu'un qui regarde des films pour se détendre.

Avec notre application, on voit les opinions de nos vrais amis. Et comme on a souvent des goûts ou des centres d'intérêt en commun, leurs avis sont beaucoup plus pertinents pour choisir quoi regarder.

Liste des modules et répartition des tâches

Nous allons proposer aux utilisateurs d'importer une image png depuis leur téléphones afin que chaque utilisateur ait une photo de profile.

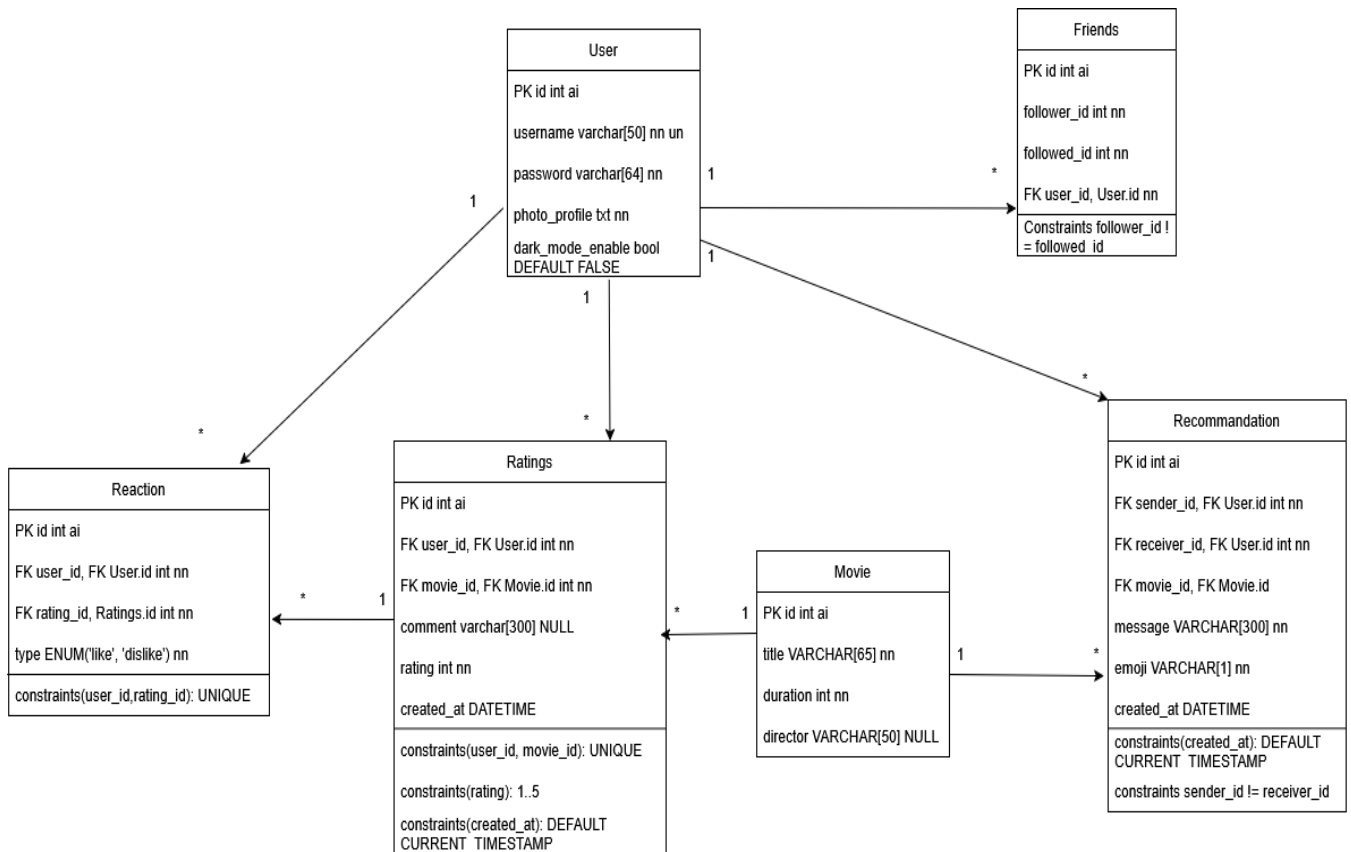
De plus, nous allons proposer un dark mode afin de rendre l'application visuellement agréable aux utilisateurs qui préfère la noirceur.

Repartitions des taches :

Gabriel.R : Importation d'un png, user, friends, recommandation

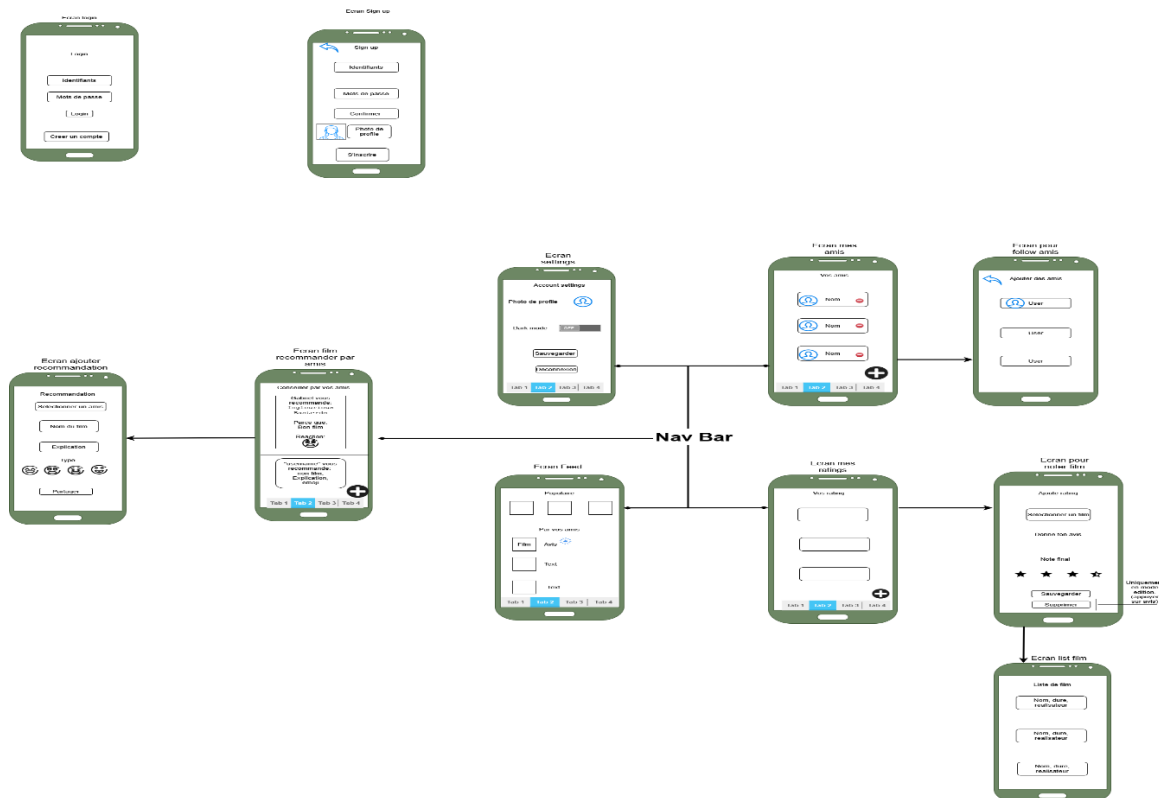
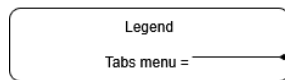
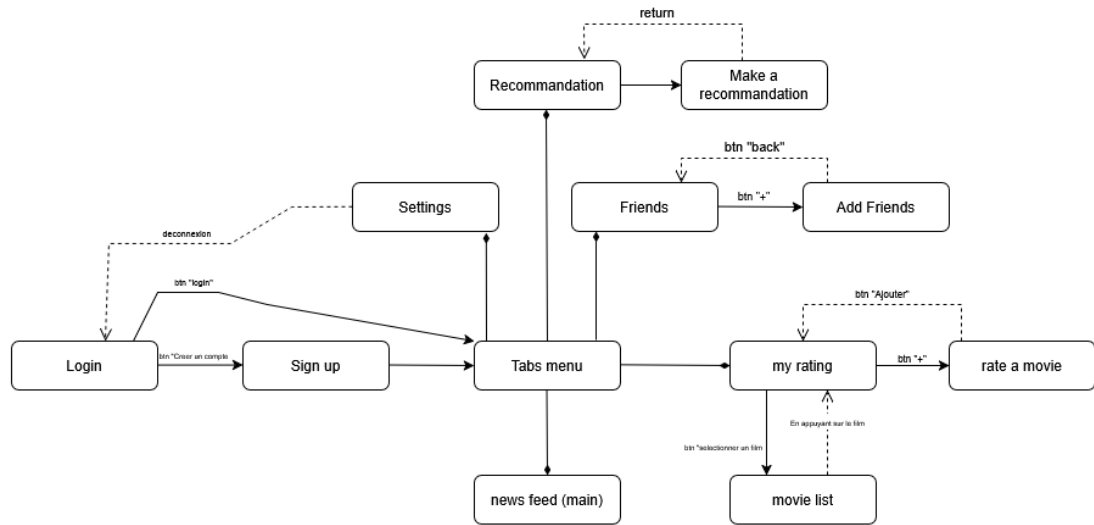
Gabriel.B : dark mode, movie, rating, reaction, nav-bar

MRD



Plus clair sur github

Storyboard



Procédurier

Ce document décrit les procédures de test et les comportements attendus pour chaque écran de l'application.

1. Authentification

Connexion (Login)

- **Action :** Clic sur "**Se connecter**" avec champs vides.
 - **Résultat :** Erreur "Veuillez remplir tous les champs."
- **Action :** Clic sur "**Se connecter**" avec nom ou mot de passe invalide.
 - **Résultat :** Erreur "Nom d'utilisateur ou mot de passe incorrect."
- **Action :** Clic sur "**Se connecter**" avec identifiants valides.
 - **Résultat :** Navigation vers l'écran **Feed** (Accueil). L'utilisateur est stocké en session.
- **Action :** Clic sur "**Créer un compte**".
 - **Résultat :** Navigation vers l'écran **Inscription**.

Inscription (Sign Up)

- **Action :** Clic sur "**S'inscrire**" avec champs vides.
 - **Résultat :** Erreur "Veuillez remplir tous les champs."
- **Action :** Clic sur "**S'inscrire**" avec mots de passe différents.
 - **Résultat :** Erreur "Les mots de passe ne correspondent pas."
- **Action :** Clic sur "**S'inscrire**" avec formulaire valide.
 - **Résultat :** Succès, compte créé et redirection vers l'écran **Home**.
- **Action :** Clic sur "**Déjà un compte ?**".
 - **Résultat :** Retour à l'écran de **Connexion**.
- **Action :** Clic sur l'icône caméra ou le conteneur avatar.
 - **Résultat :** Une photo de profil aléatoire est générée et affichée.

2. Feed (Écran Principal)

Consultation

- **Au chargement :**
 - Affichage de 3 films populaires en haut (images/titres).
 - Affichage de la liste des avis. Chaque carte affiche la photo de profil réelle de l'utilisateur (ou une initiale si absente).
 - Chaque carte affiche : Avatar, Nom, Date, Film, Note (Étoiles), Commentaire.

Interactions

- **Like (Pouce haut) :**
 - Clic : L'icône devient colorée (bleu), le compteur augmente de +1.
 - Si déjà liké : Le like est retiré, le compteur diminue de -1.
 - Si déjà disliké : Le dislike est retiré (-1), le like est ajouté (+1).
 - **Persistence :** La réaction est enregistrée en base de données. Si l'utilisateur quitte et revient (ou se reconnecte), sa réaction est conservée.
- **Dislike (Pouce bas) :**
 - Fonctionnement inverse du Like (Rouge).
 - **Persistence :** La réaction est enregistrée en base de données. Si l'utilisateur quitte et revient (ou se reconnecte), sa réaction est conservée.

3. Recommandation

Consultation

- **Au chargement :** Affichage de la liste des recommandations reçues.
- **Item :** Doit afficher "Qui recommande", "Quel film", "Raison" et "Réaction (Emoji)".

Ajout (Envoyer une recommandation)

- **Action :** Clic sur **(+)** (FAB).
 - **Résultat :** Ouverture de l'écran **"Ajouter une Recommandation"**.
- **Action :** Saisie "Sélection Ami".
 - **Résultat :** Ouvre un menu déroulant listant les amis suivis.
- **Action :** Clic sur **"Partager"** avec champs incomplets.
 - **Résultat :** Erreur "Veuillez remplir tous les champs !"
- **Action :** Clic sur **"Partager"** avec formulaire valide.
 - **Résultat :** (Mode Consultation) Affiche une alerte résumant l'envoi, puis retour.

4. Mes Ratings (Mes Films)

- **Ajout / Modification**
 - **Action :** Clic sur "Sélectionner un film".
 - **Résultat :** Navigation vers "Liste de films". Au clic sur un film, retour automatique au formulaire avec le film sélectionné (sans empiler les écrans).
 - **Action :** Clic sur "Sauvegarder".
 - **Résultat :**
 - Si nouvel avis : Création en base de données (INSERT).
 - Si avis existant : Mise à jour en base de données (UPDATE).
 - Navigation : Redirection forcée vers l'écran "Mes Ratings".
- **Suppression**
 - **Action :** Clic sur "Supprimer" (Mode Édition seulement).
 - **Résultat :** Alerte de confirmation. Si confirmé, l'avis est supprimé de la base de données (DELETE) et l'utilisateur est redirigé vers "Mes Ratings".

5. Follow (Mes Amis)

- **Suppression d'ami**
 - **Action** : Clic sur l'icône "Croix rouge" d'un ami.
 - **Résultat** : Alerte de confirmation. Si confirmé, le lien est supprimé en base de données et la liste se rafraîchit automatiquement.

5. Follow (Mes Amis)

Consultation

- **Au chargement** : Affichage de la liste des personnes suivies par l'utilisateur.
- **Item** : Avatar (Initiale) et Nom d'utilisateur.
- **Suppression** : Icône "Croix rouge" présente mais inactive (lecture seule).

Ajout d'ami

- **Action** : Clic sur (+) (FAB).
 - **Résultat** : Ouverture de l'écran "**Ajouter des Amis**".
- **Action** : Recherche (Saisie texte).
 - **Résultat** : La liste filtre les utilisateurs en temps réel via l'API.
- **Action** : Clic sur le bouton d'action (Utilisateur non suivi).
 - **Résultat** : Icône devient "Check" (Suivi ajouté).
- **Action** : Clic sur le bouton d'action (Utilisateur déjà suivi).
 - **Résultat** : Icône devient "Ajouter" (Suivi retiré).

6. Settings (Paramètres)

- **Photo de profil**
 - **Action** : Clic sur l'avatar ou le texte "Changer votre photo".
 - **Résultat** : Une nouvelle photo aléatoire est générée localement.
- **Dark Mode**
 - **Action** : Switch On/Off.

- **Résultat :** Le thème de l'application change instantanément (couleurs de fond, textes, barres de navigation).
- **Sauvegarde**
 - **Action :** Clic sur "Sauvegarder".
 - **Résultat :** L'état du Dark Mode et la nouvelle URL de la photo de profil sont enregistrés en base de données. Ces choix persisteront à la prochaine connexion.

7. Usagers de Test

- Sam | 123456
- Sophie | 123456
- Joe | 123456

Standards de Développement

1. Interface Utilisateur (UI) & Expérience (UX)

Palette de Couleurs

Nous utilisons une palette de couleurs simple et sobre afin d'avoir un look propre et professionnel. Nous utilisons un système de thème dynamique (Clair/Sombre) pour assurer un confort visuel optimal.

Couleurs Sémantiques : Les couleurs ne doivent jamais être utilisées "en dur" (hardcoded). Elles doivent provenir du contexte de thème.

Palette de base (Mode Clair par défaut) :

- **Primaire** : #4A6572 (Bleu foncé) - Boutons, titres, navigation active.
- **Arrière-plan** : #F5F5F5 (Gris très clair).
- **Surface (Cartes)** : #FFFFFF (Blanc).
- **Texte** : #333333 (Gris foncé/Noir).

Palette Mode Sombre :

- **Arrière-plan** : #121212.
- **Surface (Cartes)** : #1E1E1E.
- **Texte** : #FFFFFF.

Interaction :

- **Jaune** : #FFD700 (Étoiles).
- **Rouge** : #D32F2F / #EF5350 (Erreur, Dislike, Delete).
- **Vert** : #28A745 (Succès, Ajout).

Composants Visuels Standards

- **Cartes (Cards)** :

- Bords arrondis
- Ombres portées : Légère élévation
- **Boutons (Buttons) :**
 - Forme : Arrondie
 - Style : Fond plein (PRIMARY_COLOR) pour l'action principale, Bordure seule pour l'action secondaire.
- **Avatars (Profile Picture) :**
 - **Priorité** : Affichage de l'image via URL (photo_profile).
 - **Fallback** : Si aucune image, affichage des initiales de l'utilisateur dans un cercle de couleur Primaire.
- **Listes** : Utilisation systématique de FlatList.

Navigation

- **Type** : Bottom tab et stack.
- **Bottom tab** : Tab bar standard avec React Navigation.

2. Code & Architecture (React Native)

Structure des Fichiers

- Organisation par fonctionnalité.

Nomenclature des Fichiers (Naming Convention)

- **Écrans** : camelCase avec suffixe _screen.
 - Exemple : feed_screen.js, login_screen.js.
- **Services & Contextes** : camelCase.
 - Exemple : api.js, AuthContext.js.

Conventions de Code JS

- **Composants** : Fonctions fléchées ou export default function Name().
Toujours en PascalCase.

- **Variables & Fonctions** : camelCase (ex: handleLogin, fetchData).

Gestion d'État

- **Local** : useState pour les formulaires, les chargements (isLoading), et les données spécifiques à un écran.
- **Global** : Context API (AuthContext) pour stocker l'objet user connecté et le rendre accessible partout via le hook useAuth().

Effets de bord

- useEffect pour le chargement initial.
- useFocusEffect (de React Navigation) pour rafraîchir les données quand on revient sur un écran.

3. Base de Données

Nomenclature SQL

- **Tables** : PascalCase (Singulier).
- **Colonnes** : snake_case.
- **Clés Primaires** : Toujours nommées id (INT AUTO_INCREMENT).
- **Clés Étrangères** : Format [table]_id (ex: follower_id référence User.id).

Structure des Tables Clés

- **User** : Identifiants, mot de passe, photo.
- **Movie** : Titre, durée, réalisateur (Données statiques pour le moment).
- **Ratings** : Lien User-Movie avec note (1-5) et commentaire. Contrainte UNIQUE(user_id, movie_id).
- **Follow** : Relation User-User. Contrainte CHECK (follower != followed).
- **Reaction** : 'like' ou 'dislike' sur un Rating.
- **Recommendation** : Message envoyé d'un User à un autre pour un Movie spécifique.

4. Communication API (Martha)

Gestion des Requêtes

- **Centralisation** : Tous les appels passent par la fonction `executeQuery` dans `services/api.js`.
- **Définition** : Les requêtes SQL brutes sont stockées sur **Martha** et appelées par leur nom.

Liste des Requêtes

- `login_user`, `create_user` (Auth)
- `get_feed` (Lecture principale)
- `get_my_ratings`, `create_rating` (Gestion avis)
- `get_my_friends`, `follow_user`, `unfollow_user`, `search_users` (Social)
- `add_reaction`, `remove_reaction` (Interactions)
- `send_recommendation`, `get_my_recommendations` (Partage)

Guide de Déploiement et Sauvegarde

Étapes nécessaires pour restaurer un fonctionnement minimal de l'application dans un nouvel environnement (nouvelle base de données Martha, nouveau poste de développement).

ÉTAPE 1 : La Base de Données (Juste le nécessaire)

On va créer seulement les tables User et Movie pour commencer.

1. Allez dans **Martha** > Onglet **SQL**.
2. Copiez ceci et cliquez sur **Exécuter** :

SQL

```
SET FOREIGN_KEY_CHECKS = 0;
```

```
DROP TABLE IF EXISTS Movie;
```

```
DROP TABLE IF EXISTS User;
```

-- 1. Table Utilisateur

```
CREATE TABLE User (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    username VARCHAR(50) NOT NULL UNIQUE,  
    password VARCHAR(255) NOT NULL,  
    photo_profile TEXT NOT NULL,  
    dark_mode_enabled BOOLEAN DEFAULT FALSE  
);
```

-- 2. Table Film

```
CREATE TABLE Movie (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    title VARCHAR(150) NOT NULL,
```

```
duration INT NOT NULL,  
director VARCHAR(50)  
);
```

```
SET FOREIGN_KEY_CHECKS = 1;
```

-- 3. Données de test (1 user, 2 films)

```
INSERT INTO User (username, password, photo_profile) VALUES  
('Sophie', '123456', 'https://randomuser.me/api/portraits/women/44.jpg');
```

```
INSERT INTO Movie (title, duration, director) VALUES  
('Dune: Part Two', 166, 'Denis Villeneuve'),  
('Oppenheimer', 180, 'Christopher Nolan');
```

ÉTAPE 2 : L'API (Juste 2 requêtes)

Dire à Martha comment connecter l'utilisateur et comment récupérer les films.

1. Allez dans **Martha** > Bouton **Import JSON**.
2. Copiez ceci et validez :

JSON

```
[  
  {  
    "name": "login_user",  
    "string": "SELECT * FROM User WHERE username=\"?username\" AND  
password=\"?password\";",  
    "type": 0  
  },  
]
```

```
{  
  "name": "get_all_movies",  
  "string": "SELECT * FROM Movie ORDER BY title ASC;",  
  "type": 0  
}  
]
```

ÉTAPE 3 : Connecter le Code

S'assurer que le fichier services/api.js contient la bonne clé pour le compte Matha .

1. Ouvrez services/api.js.
2. S'assurer que la clé utiliser est la bonne.

ÉTAPE 4 : Lancer l'application

1. Dans le terminal, Lancez : npx expo start
2. Dans l'application :
 - **Login** : Sophie | 123456.
 - **Test** : Une fois connecté, sur l'écran MovieList. Il faut voir "Dune" et "Oppenheimer"

Ajout du module Ratings

Cette section permet d'activer la création, la lecture, la modification et la suppression d'avis sur les films.

ÉTAPE 1 : Ajouter la Table Ratings Allez dans Martha > Onglet SQL. Exécutez ce code pour créer la table qui lie les utilisateurs aux films avec une note :

```
SET FOREIGN_KEY_CHECKS = 0;
```

```
DROP TABLE IF EXISTS Ratings;
```

```

CREATE TABLE Ratings (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  movie_id INT NOT NULL,
  comment VARCHAR(300),
  rating INT NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (user_id) REFERENCES User(id) ON DELETE CASCADE,
  FOREIGN KEY (movie_id) REFERENCES Movie(id) ON DELETE CASCADE,

  UNIQUE(user_id, movie_id),
  CHECK (rating >= 1 AND rating <= 5)
);

SET FOREIGN_KEY_CHECKS = 1;

```

ÉTAPE 2 : Ajouter les requêtes API pour les Ratings Il faut dire à Martha comment gérer les notes. Allez dans Martha > Bouton Import JSON. Copiez ce bloc (qui inclut Création, Lecture, Modification et Suppression) :

```

[
  {
    "name": "create_rating",
    "string": "INSERT INTO Ratings (user_id, movie_id, rating, comment) VALUES
(?user_id, ?movie_id, ?rating, \"?comment?\");",
    "type": 1
  }
]

```

```

},
{
  "name": "get_my_ratings",

  "string": "SELECT r.id, r.rating, r.comment, r.created_at, m.id as movie_id, m.title
as movie_title, m.duration, m.director FROM Ratings r JOIN Movie m ON r.movie_id =
m.id WHERE r.user_id=?user_id ORDER BY r.created_at DESC;",

  "type": 0
},
{
  "name": "update_rating",

  "string": "UPDATE Ratings SET rating=?rating, comment=?comment\" WHERE
id=?id AND user_id=?user_id;",

  "type": 1
},
{
  "name": "delete_rating",

  "string": "DELETE FROM Ratings WHERE id=?id AND user_id=?user_id;",

  "type": 1
}
}
]

```

ÉTAPE 3 : Tester le Rating

1. Dans l'application, connectez-vous (Sophie | 123456).
2. Allez dans l'onglet **"Mes ratings"**.
3. Appuyez sur le bouton **"+"** (en bas à droite).
4. Cliquez sur **"Sélectionner un film"** et choisissez *"Dune: Part Two"*.
5. Donnez une note (ex: 5 étoiles) et un commentaire.

6. Appuyez sur "**Sauvegarder**".
7. **Résultat attendu** : Vous revenez à la liste et votre avis sur Dune est affiché.